TESI 2105 · Programación Orientada a Objetos

Unidad E

Punteros

Direcciones de memoria, arreglos y funciones



Prof. Eduardo O. De Jesús Arroyo
Instituto Tecnológico de Puerto Rico · Recinto de Manatí

Objetivos de la unidad

Al finalizar, el estudiante podrá…



Explicar qué es un puntero

y usar los operadores & (dirección) y * (contenido).



Recorrer arreglos con punteros

aplicando la aritmética de punteros: p + 1, p++, p2 − p1.



Pasar punteros a funciones

para modificar variables y arreglos del programa principal.



Usar punteros a punteros

y reconocer los errores más comunes con la memoria.

Mapa de la unidad

Siete temas del prontuario, en orden

1. **Concepto de punteros**
2. **Declaración de punteros**
3. **Punteros y arreglos**
4. **Aritmética de punteros**
5. **Punteros como parámetros**
6. **Paso de punteros a funciones**
7. **Punteros a punteros**

Cerramos con errores comunes, repaso y el Laboratorio E.

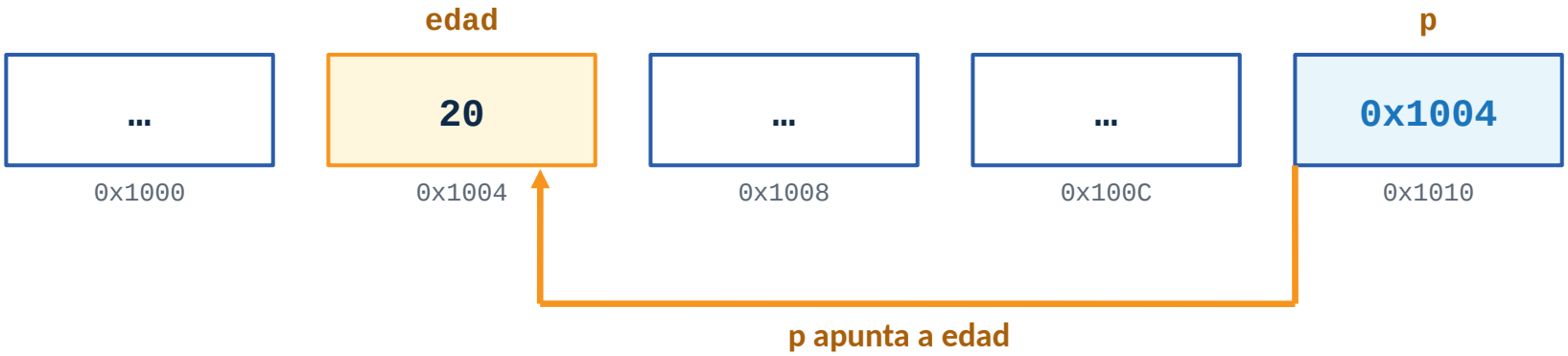


Tema 1 de 7

Concepto de punteros

La memoria es una fila de casillas numeradas

Cada variable vive en una dirección; un puntero guarda esa dirección



Un puntero es una variable cuyo valor es una **dirección de memoria**. No guarda el dato: guarda dónde está el dato. Las direcciones del dibujo son de ejemplo; las reales las asigna el sistema y cambian en cada ejecución.

Dos operadores para trabajar con direcciones

Uno pregunta «¿dónde está?» y el otro «¿qué hay allí?»

`&` dirección de

`&edad`

Devuelve la dirección donde está guardada la variable.

```
int* p = &edad;
```

`*` contenido de

`*p`

Va a la dirección guardada en p y lee o cambia el valor que hay allí (desreferenciar).

```
*p = 21;   // edad vale 21
```



Tema 2 de 7

Declaración de punteros

Cómo se declara un puntero

tipo nombre; — el tipo dice a qué clase de dato apunta*

Declaración	Se lee
`int* p;`	p apunta a un int
`double* q;`	q apunta a un double
`char* c;`	c apunta a un char
`int* p = nullptr;`	p no apunta a nada todavía

Regla de oro: inicialice siempre el puntero. Si todavía no tiene a dónde apuntar, use nullptr.

El tipo tiene que coincidir:

```
double precio = 9.75;
int* p = &precio;
```

```
error: cannot convert 'double*' to 'int*' in initialization
```

Mensaje literal de g++

Primer programa con un puntero

Cambiar una variable sin usar su nombre

e1.cpp · compilado con -Wall

```
int edad = 20;
int* p = &edad;   // p guarda la direccion

cout << "edad = " << edad << endl;
cout << "*p   = " << *p << endl;

*p = 21;          // cambia edad a traves de p
cout << "edad = " << edad << endl;
```

Salida

```
edad = 20
*p   = 20
edad = 21
```

Nunca escribimos edad = 21. El cambio llegó por la dirección.

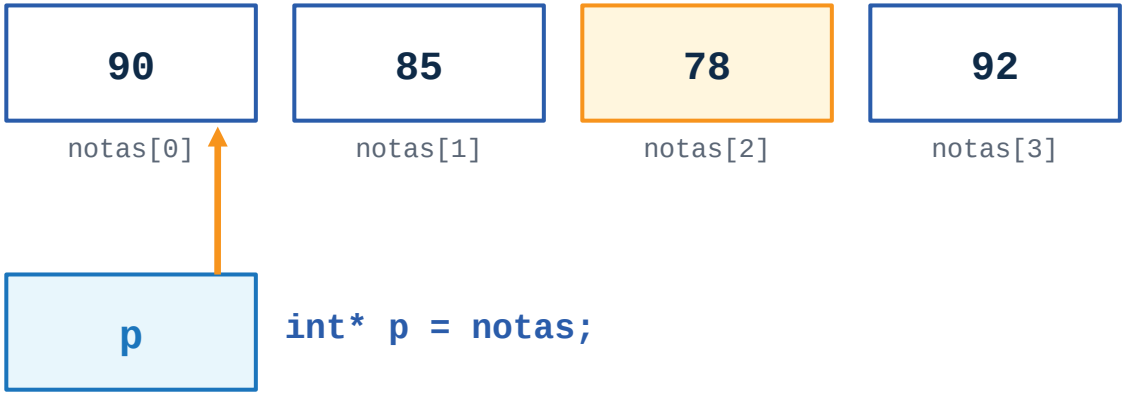


Tema 3 de 7

Punteros y arreglos

El nombre del arreglo es una dirección

notas equivale a &notas[0]: la dirección del primer elemento



Las tres formas leen el mismo valor: 78		
`notas[2]`	`*(p + 2)`	`p[2]`

Diferencia clave

El puntero p se puede mover por el arreglo. El nombre notas no: siempre señala al inicio.



Tema 4 de 7

Aritmética de punteros

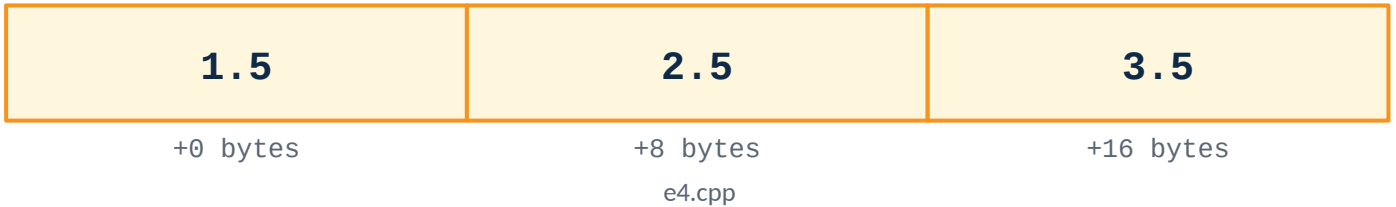
p + 1 avanza un elemento, no un byte

El compilador multiplica por el tamaño del tipo

int (4 bytes)



double (8 bytes)



e4.cpp

```
int* p = a;   double* q = d;
int* fin = a + 3;
for (int* r = a; r < a + 3; r++)
    cout << *r << " ";
```

Salida

```
int:    p+1 avanza 4 bytes
double: q+1 avanza 8 bytes
fin - a = 3 elementos
10 20 30
```

Qué operaciones se permiten

Solo las que tienen sentido como «moverse por el arreglo»

Válidas

`p + n,  p - n`	Avanzar o retroceder n elementos
`p++,  p--`	Mover el puntero uno a uno
`p2 - p1`	Distancia en elementos (mismo arreglo)
`p < a + n`	Comparar para saber si llegó al final

No válidas o peligrosas

`p1 + p2`	Sumar dos direcciones no significa nada
`p * 2,  p / 2`	No se multiplica ni divide un puntero
`p + n  fuera`	Pasarse del arreglo: comportamiento indefinido



Tema 5 de 7

Punteros como parámetros

Por valor contra por puntero

Solo el puntero deja que la función cambie la variable de main()

e5.cpp

```
void duplicarValor(int x)    { x = x * 2; }
void duplicarPuntero(int* x) { *x = *x * 2; }

int n = 5;
duplicarValor(n);       // recibe una copia
duplicarPuntero(&n);    // recibe la direccion
```

Salida

```
Por valor:   5
Por puntero: 10
```

Por valor: La función trabaja con una copia. Al terminar, la copia desaparece y n sigue en 5.

Por puntero: La función recibe la dirección de n y escribe allí con *x. El cambio queda.

Puntero (*) y referencia (&) de la Unidad D

Las dos permiten modificar el original; se escriben distinto

	Referencia int& x	Puntero int* x
Llamada	`duplicar(n);`	`duplicar(&n);`
Dentro de la función	`x = x * 2;`	`*x = *x * 2;`
¿Puede ser nulo?	No	Sí (nullptr)
¿Puede cambiar a dónde apunta?	No	Sí
¿Recorre arreglos?	No	Sí, con aritmética

En C++ moderno se prefiere la referencia cuando basta; el puntero se usa cuando hace falta lo que solo él puede hacer.



Tema 6 de 7

Paso de punteros a funciones

Un arreglo viaja a la función como un puntero

La función recibe la dirección del primer elemento, por eso necesita n

e6.cpp

```
double promedio(const int* a, int n) {
    int suma = 0;
    for (int i = 0; i < n; i++)
        suma += *(a + i);
    return (double)suma / n;
}

promedio(notas, 4);    // 86.25
```

const protege los datos

const int* a: la función puede leer el arreglo, pero no cambiarlo.

Si lo intenta, g++ lo detiene:

```
a[0] = 0;
```

```
error: assignment of read-only
location '* a'
```

Mensaje literal de g++

Una función también puede devolver un puntero

buscarMayor devuelve dónde está el mayor, no solo cuánto vale

e6.cpp

```
int* buscarMayor(int* a, int n) {
    int* mayor = a;
    for (int* p = a + 1; p < a + n; p++)
        if (*p > *mayor) mayor = p;
    return mayor;
}

int* m = buscarMayor(notas, 4);
cout << *m << " en la posicion " << m - notas;
```

Salida

```
Mayor: 92 en la posicion 3
```

Con la dirección se obtienen dos datos:
`*m` el valor
`m - notas` la posición

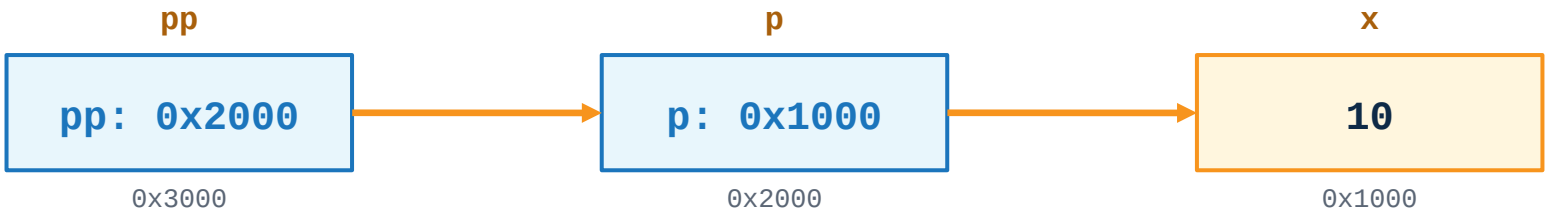


Tema 7 de 7

Punteros a punteros

Un puntero que guarda la dirección de otro puntero

*int** pp: dos saltos para llegar al dato*



e7.cpp

```
int x = 5;
int* p = &x;
int** pp = &p;
**pp = 10;      // dos saltos: pp -> p -> x
```

Salida

```
x = 10, *p = 10, **pp = 10
```

¿Para qué sirve un puntero a puntero?

Para que una función cambie HACIA DÓNDE apunta un puntero de main()

Laboratorio E

```
void marcarDestacado(int** destacado, int* cant, int pos) {
    *destacado = cant + (pos - 1);
}

int* destacado = nullptr;
marcarDestacado(&destacado, cant, 2);
// ahora destacado apunta a cant[1]
```

Con int* la función solo cambia el dato. Con int** cambia el puntero mismo.

Lo verá otra vez: char** argv en main(int argc, char** argv), y en la memoria dinámica de la Unidad H.

Errores comunes con punteros

El compilador no siempre los detecta



Desreferenciar nullptr

*p cuando p es nullptr detiene el programa (segmentation fault).



Puntero sin inicializar

int* p; apunta a cualquier lugar. Inicialice siempre.



Salirse del arreglo

*(p + 4) en un arreglo de 4 lee memoria ajena.



Confundir * y &

Pasar n cuando la función espera &n, o olvidar el * al asignar.

Herramienta: AddressSanitizer

Compile con -fsanitize=address mientras practica

e8.cpp

```
int notas[4] = {90, 85, 78, 92};
int* p = notas;
return *(p + 4);   // fuera del arreglo
```

```
g++ -g -fsanitize=address e8.cpp -o e8
./e8
```

Disponible en g++ de Linux y macOS. En Windows, use OnlineGDB o WSL para esta prueba.

Salida

```
ERROR: AddressSanitizer:
stack-buffer-overflow
#0 in main e8.cpp:4
```

Con -Wall solo aparece una advertencia que no explica el problema. AddressSanitizer dice qué pasó y en qué línea.

Conexión con la programación orientada a objetos

Los punteros son la base de lo que viene en las Unidades H e I



`this`

Dentro de una clase, this es un puntero al objeto que ejecuta el método.



`new y delete`

Los objetos creados en tiempo de ejecución se manejan con punteros.



`Polimorfismo`

Un puntero a la clase base puede señalar a un objeto de una clase derivada.

Repaso rápido

Responda antes del laboratorio

1. Si int* p = &n; ¿qué diferencia hay entre p y *p?

2. Un int ocupa 4 bytes. Si p vale 0x1000, ¿cuánto vale p + 3?

3. ¿Por qué una función que recibe un arreglo necesita también su tamaño?

4. ¿Qué se necesita para que una función cambie a dónde apunta un puntero de main()?



Laboratorio E

Inventario del Laboratorio A-201

- Recorrer el inventario con aritmética de punteros
- Sumar unidades y calcular el valor total
- Buscar el equipo con más unidades (función que devuelve un puntero)
- Recibir equipo e intercambiar posiciones (punteros como parámetros)
- Marcar un equipo destacado (puntero a puntero)

Referencias

cppreference.com. (s. f.). Pointer declaration. https://en.cppreference.com/w/cpp/language/pointer

Kumar, A. (2024). Hands-on object-oriented programming: Mastering OOP features for real-world software systems development. Apress.

Lutz, M. (2025). Learning Python: Powerful object-oriented programming. O’Reilly Media.

Tsetsekas, H. (2023). Object-oriented programming exercises with C++. Publicación independiente.

Todos los ejemplos se compilaron y ejecutaron con g++ -std=c++17 -Wall -Wextra.